2520030117

PRACTICAL 8

1. Dynamic Memory Allocation using `malloc()`, `calloc()`, `realloc()` and `free()`

AIM

To develop a C program using `malloc()`, `calloc()`, `realloc()`, and `free()` to demonstrate dynamic memory allocation and deallocation, and to monitor memory allocation behavior and identify memory leaks using **Valgrind**.

REQUIREMENTS

- Ubuntu/Linux operating system
- GCC compiler
- Valgrind
- C programming environment / VS Code

THEORY

Dynamic memory allocation allows a program to allocate memory during runtime.

The following functions are used:

- **`malloc()`** – Allocates a block of uninitialized memory.
- **`calloc()`** – Allocates memory for multiple elements and initializes it to zero.
- **`realloc()`** – Changes the size of an already allocated memory block. ● **`free()`** – Releases dynamically allocated memory.

Valgrind is a memory debugging tool used to detect:

- Memory leaks
- Invalid memory accesses
- Incorrect memory deallocation

ALGORITHM

1. Start the program.
2. Allocate memory for an integer array using `malloc()`.
3. Store and display values in the allocated memory.
4. Allocate another integer array using `calloc()`.
5. Display the zero-initialized values.
6. Increase the size of the `malloc()` allocated array using `realloc()`.

7. Store additional values and display the updated array.
8. Release allocated memory using `free()`.
9. Stop the program.
10. Run the program using Valgrind to check for memory leaks.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    printf("Dynamic Memory Allocation Demonstration\n\n");

    // malloc()
    int *arr = (int *)malloc(3 * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation using malloc() failed.\n");
        return 1;
    }

    printf("Memory allocated using malloc():\n");

    for (i = 0; i < 3; i++)
    {
        arr[i] = (i + 1) * 10;
        printf("%d ", arr[i]);
    }

    printf("\n");

    // calloc()
    int *zero_arr = (int *)calloc(3, sizeof(int));

    if (zero_arr == NULL)
    {
        printf("Memory allocation using calloc()
        failed.\n"); free(arr);
        return 1;
    }

    printf("\nMemory allocated using calloc():\n");

    for (i = 0; i < 3; i++)
    {
```

```
        printf("%d ", zero_arr[i]);
    }

    printf("\n");

    // realloc()
    arr = (int *)realloc(arr, 5 * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory reallocation failed.\n");
        free(zero_arr);
        return 1;
    }

    arr[3] = 40;
    arr[4] = 50;

    printf("\nAfter realloc(), updated array:\n");

    for (i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    // free()
    free(arr);
    free(zero_arr);

    printf("\nMemory successfully released using free().\n");

    return 0;
}
```

COMPILATION AND EXECUTION

Save the file as:

memory_allocation.c

Compile:

```
gcc memory_allocation.c -o memory_allocation
```

Run:

```
./memory_allocation
```

EXPECTED OUTPUT

Dynamic Memory Allocation Demonstration

Memory allocated using malloc():
10 20 30

Memory allocated using calloc():
0 0 0

After realloc(), updated array:
10 20 30 40 50

Memory successfully released using free().

VALGRIND MEMORY LEAK DETECTION

Run:

valgrind --leak-check=full ./memory_allocation

EXPECTED VALGRIND RESULT

At the end, you should see something similar to:

HEAP SUMMARY:
in use at exit: 0 bytes in 0 blocks
total heap usage: ... allocs, ... frees, ... bytes allocated

All heap blocks were freed -- no leaks are possible

ERROR SUMMARY: 0 errors from 0 contexts

OBSERVATION

The program demonstrates all four dynamic memory functions:

Function	Purpose
`malloc( )`	memory Allocates and
`calloc( )`	initializes memory to zero
`realloc ()`	Allocates uninitialized

Resizes previously	allocated memory

`free()`	Releases allocated memory

Valgrind confirms that all dynamically allocated memory has been properly released.

RESULT

Thus, the C program successfully demonstrated dynamic memory allocation using `malloc()`, `calloc()`, `realloc()`, and `free()`. The allocated memory was properly released, and Valgrind was used to verify that there were **no memory leaks**.

2. Demonstration of Copy-on-Write after `fork()`

AIM

To develop a C program that demonstrates **Copy-on-Write (COW)** behavior after `fork()` and analyze memory usage before and after modifying data in the child process.

REQUIREMENTS

- Ubuntu/Linux operating system
- GCC compiler
- C programming environment / VS Code
- Linux `fork()` system call

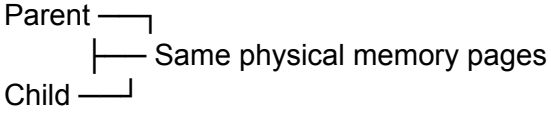
THEORY

When a process calls `fork()`, the operating system creates a child process.

Initially, the parent and child processes **share the same physical memory pages**. They do not immediately receive separate copies of all the memory.

This is called **Copy-on-Write (COW)**.

If the child only reads the data:



When the child modifies a shared page, the operating system creates a **private copy of that page** for the child.

Parent ──> Original page
Child ──> Copied page

Therefore, memory usage increases when the child modifies the shared data.

ALGORITHM

1. Start the program.
2. Allocate a large block of memory dynamically.
3. Initialize the allocated memory.
4. Display the memory usage of the parent process.
5. Call `fork()` to create a child process.
6. The parent and child initially share the memory pages using Copy-on-Write.
7. Display the child's memory usage before modification.
8. Modify the allocated data in the child process.
9. The operating system creates private copies of modified memory pages.
10. Display the child's memory usage after modification.
11. Parent waits for the child process to finish.
12. Release the allocated memory.
13. Stop the program.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (50 * 1024 * 1024)

void show_memory(const char *message)
{
    FILE *file;
    char line[256];

    file = fopen("/proc/self/status", "r");

    if (file == NULL)
    {
        perror("fopen");
        return;
    }

    printf("\n%s\n", message);
```

```
    while (fgets(line, sizeof(line), file))
    {
        if (strncmp(line, "VmRSS:", 6) == 0)
        {
            printf("%s", line);
            break;
        }
    }

    fclose(file);
}

int main()
{
    char *memory;

    memory = (char *)malloc(SIZE);

    if (memory == NULL)
    {
        printf("Memory allocation
        failed.\n"); return 1;
    }
    for (int i = 0; i < SIZE; i++)
    {
        memory[i] = 0;
    }

    printf("Parent process started.\n");

    show_memory("Parent memory before fork:");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(memory);
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild process created.\n");

        show_memory("Child memory before modification:");
```

```
        printf("Child is modifying the shared memory...\n");

        for (int i = 0; i < SIZE; i += 4096)
        {
            memory[i] = 1;
        }

        show_memory("Child memory after modification:");

        free(memory);
        exit(0);
    }
    else
    {
        wait(NULL);

        printf("\nParent process: Child completed.\n");

        show_memory("Parent memory after child

        modification:");

        free(memory);
    }

    return 0;
}
```

⚠️ **One tiny correction:** because this program uses `strncmp()`, add this header at the top:

```
#include <string.h>
```

So your final header section should be:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
```

COMPILATION AND EXECUTION

Save as:

copy_on_write.c

Compile:

```
gcc copy_on_write.c -o copy_on_write
```

Run:

./copy_on_write

EXPECTED OUTPUT

The exact memory values can differ depending on the system, but the output will look similar to:

Parent process started.

Parent memory before fork:
VmRSS: 52000 kB

Child process created.

Child memory before modification:
VmRSS: 51200 kB
Child is modifying the shared memory...

Child memory after modification:
VmRSS: 102400 kB

Parent process: Child completed.

Parent memory after child modification:
VmRSS: 52000 kB

OBSERVATION

Before modification, the parent and child can share the same physical memory pages.

When the child modifies the memory:

Child modifies shared page
↓
Operating System detects write
↓
A private copy of the page is created
↓
Child uses the copied page

Thus, the program demonstrates the basic principle of **Copy-on-Write**.

RESULT

Thus, the program successfully demonstrated **Copy-on-Write behavior after `fork()`**. The parent and child initially share memory pages, and when the child modifies the shared data, the operating system creates private copies of the modified pages.

```
Dynamic Memory Allocation Demonstration

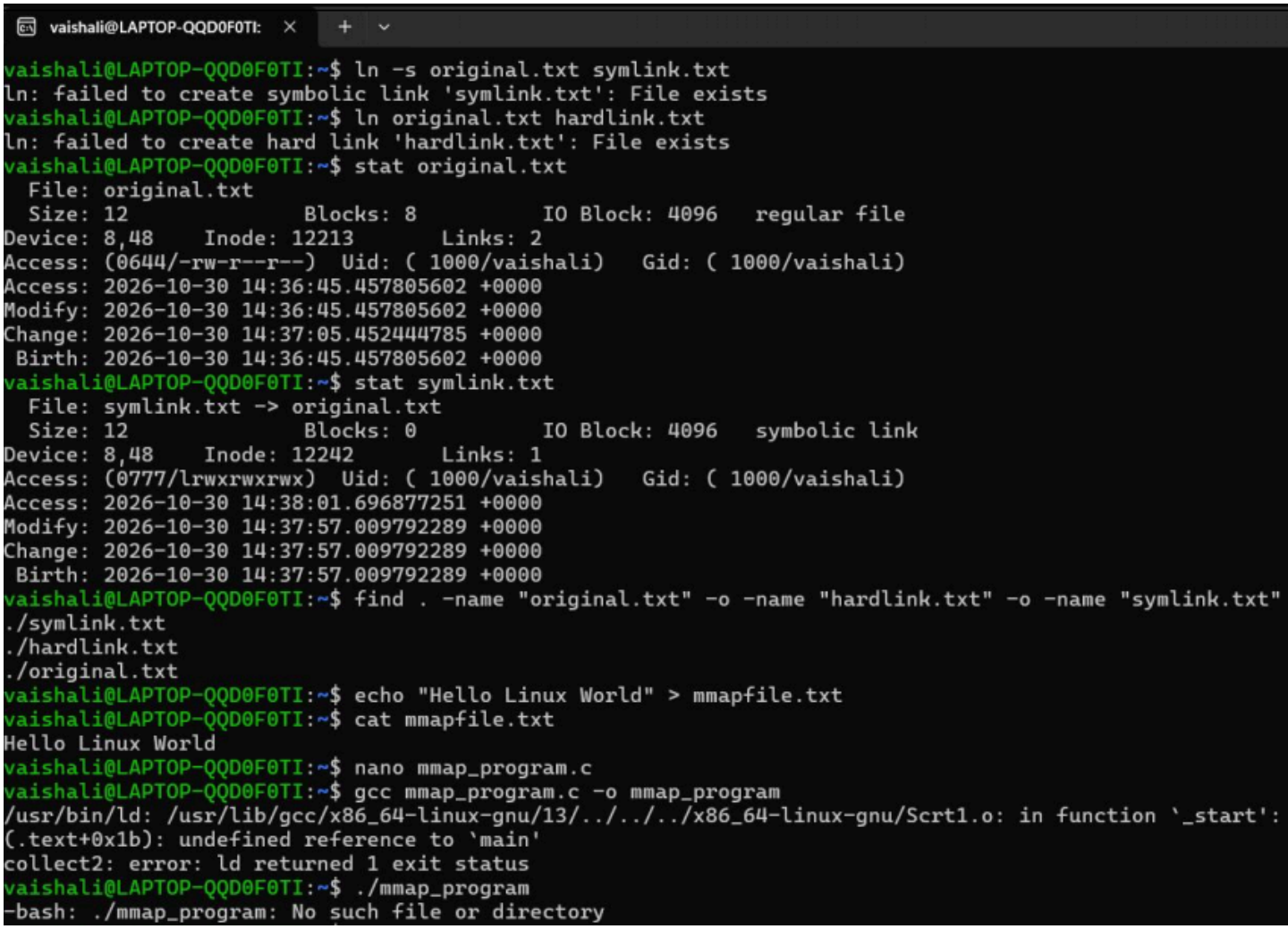
Memory allocated using malloc():
10 20 30

Memory allocated using calloc():
0 0 0

After realloc(), updated array:
10 20 30 40 50

Memory successfully released using free().
```

```
ishali@LAPTOP-QQD0F0TI:~$ mkdir -p Practical-10
ishali@LAPTOP-QQD0F0TI:~$ cd /mnt/c/Users/Vaishali/OneDrive/Desktop/OS/Practical/
ash: cd: /mnt/c/Users/Vaishali/OneDrive/Desktop/OS/Practical/Practical-10: No su
ishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux" > original.txt
ishali@LAPTOP-QQD0F0TI:~$ ls -l original.txt
w-r--r-- 1 vaishali vaishali 12 Oct 30  2026 original.txt
ishali@LAPTOP-QQD0F0TI:~$ stat original.txt
File: original.txt
Size: 12              Blocks: 8          IO Block: 4096   regular file
vice: 8,48    Inode: 12213       Links: 1
cess: (0644/-rw-r--r--)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
cess: 2026-10-30 14:36:45.457805602 +0000
dify: 2026-10-30 14:36:45.457805602 +0000
ange: 2026-10-30 14:36:45.457805602 +0000
irth: 2026-10-30 14:36:45.457805602 +0000
ishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
213 hardlink.txt  12213 original.txt
ishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
: failed to create hard link 'hardlink.txt': File exists
ishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
213 hardlink.txt  12213 original.txt
ishali@LAPTOP-QQD0F0TI:~$
```




```
10 20 30 40 50

Memory successfully released using free().
==473==
==473== HEAP SUMMARY:
==473==     in use at exit: 0 bytes in 0 blocks
==473==   total heap usage: 4 allocs, 4 frees, 1,068 bytes alloc
ated
==473==
==473== All heap blocks were freed -- no leaks are possible
==473==
==473== For lists of detected and suppressed errors, rerun with:
 -s
==473== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 f
rom 0)
```

```
Parent process started.

Parent memory before fork:
VmRSS:     52620 kB

Child process created.

Child memory before modification:
VmRSS:     52076 kB
Child is modifying the shared memory...

Child memory after modification:
VmRSS:     52076 kB

Parent process: Child completed.

Parent memory after child modification:
VmRSS:     52812 kB
```